COMPILADORES 2026.2

MiniForge

um compilador de mineirês, sô

```
trem n: inteirim = 5;
trem f: inteirim = 1;
enquanto_tiver (n > 0) {
    f = f * n;
    n = n - 1;
}
mostra_so(f);
```

Frontend até 16/10/2026
Análise semântica · aula 22

Pedro Dutra · Pedro Pignata
Lucas Rocha Dantas · Renato Portilho
prof.ª Lorena Bezerra

A linguagem própria do grupo

Mineirês	Papel
trem	Declara variável
inteirim / booleanim	Tipos int / bool
eh_memo / num_eh	Verdadeiro / falso
uai / sinao	Condicional
enquanto_tiver	Repetição
mostra_so	Comando de saída na AST

ESCOPO

O corte desta entrega

Aulas 06 a 22

Especificação, lexer, gramática, parser e AST
Checkpoint, símbolos, escopos e tipos

A CLI valida o fonte e informa sucesso.
A geração de código e a execução virão nas próximas entregas.

Arquitetura do frontend

Fonte .mini → **Lexer** → **Parser** → **Semântica**

texto → tokens → AST → AST tipada

```
tokens.py / lexer.py
parser.py / ast.py
simbolos.py / semantica.py
```

Cada fase só recebe entrada válida.

Texto e tokens

```
trem n: inteirim = 5;

2:1   VAR          'trem'
2:6   ID           'n'
2:7   DOIS_PONTOS  ':'
2:9   INT          'inteirim'
```

- Maior casamento
- Reservadas após ler o nome
- Linha e coluna desde 1
- Comentários e espaços ignorados
- EOF e recuperação de erros

```
1:3: erro léxico: caractere inválido '@', que trem é esse?
```

Gramática, parser e AST

```
mostra_so(1 + 2 * 3);

Print
└── Binario +
    ├── Num 1
    └── Binario *
        ├── Num 2
        └── Num 3
```

Parser recursivo descendente LL(1)

10 - 3 - 2 agrupa à esquerda
Parênteses alteram o agrupamento
Menos unário tem maior precedência

Chaves obrigatórias nos controles
Comparações não encadeadas

Nomes e escopos

```
trem x: inteirim = 1;
{
    trem x: booleanim = eh_memo;
    mostra_so(x);
}
mostra_so(x);
```

Dentro do bloco: x é bool
Fora do bloco: x é int

Simbolo guarda nome, tipo, categoria,
linha e nível do escopo.

O inicializador vem antes da declaração.

Redeclaração no mesmo escopo é erro. Sombreamento interno é permitido.

Regras semânticas e tipos

Operador	Operandos internos	Resultado
+ - * /	int × int	int
< > <= >=	int × int	bool
== !=	int × int ou bool × bool	bool
- unário	int	int

Inicialização e atribuição exigem o tipo declarado.
uai e enquanto_tiver exigem condição booleanim.

A AST anotada

```
mostra_so(1 + 2 < 4);

Print
└── Binario < : booleanim
    ├── Binario + : inteirim
    │   ├── Num 1 : inteirim
    │   └── Num 2 : inteirim
    └── Num 4 : inteirim
```

Expr.tipo

None após o parser
int ou bool após
a semântica válida

Diagnósticos sem cascata

```
mostra_so(z + 1 + 2);

Um erro de origem:
'z' num foi declarado, sô
```

Três erros independentes no exemplo semantico.mini

b recebe inteirim, mas foi declarado booleanim
A condição uai (x) usa inteirim
y recebe atribuição sem declaração

VALIDAÇÃO

Testes e demonstração

377 testes aprovados

42 casos de linguagem · 32 combinações de opções
Python 3.12.3 · pytest 8.4.2 · Linux

```
python -m miniforge exemplos/fatorial.mini
python -m miniforge exemplos/if_else.mini --tipos
python -m miniforge exemplos/semantico.mini --contexto
python -m pytest -q
```

Continuidade do mesmo projeto

Aula 26 · 30/10 no cronograma
Representação intermediária e código de três endereços

Aula 28 · 06/11 no cronograma
Máquina virtual e testes de execução

Aula 30 · 13/11 no cronograma
Otimizações, equivalência e medições reais